PES UNIVERSITY

BENGALURU

DEPARTMENT OF CSE (AI & ML)

COURSE: DIGITAL LOGIC AND COMPUTER DESIGN

COURSE CODE: UE25CS251A

MINI-PROJECT REPORT

Design and implement a 4-bit shift register (Parallel - In Parallel-Out and Parallel - In Serial - Out) using Verilog HDL, and verify it using a test bench.

PROJECT TEAM

SL. NO.	STUDENT NAME	SRN
01	Rahul P	PES1UG25AM510
02	Rohith M	PES1UG25AM512
03	S Banuteja Reddy	PES1UG25AM513
04	Sambhram Laxman Sattigeri	PES1UG25AM516

SUBMITTED TO

Prof. Sonali Kalekar K

Assistant Professor

Department of CSE (AI & ML)

PES University

Contents

1 Problem Statement **2**

2 Theory **2**

2.1 Shift Registers 2

2.2 PIPO Shift Register 2

2.3 PISO Shift Register 3

3 Circuit **4**

4 Truth Table **4**

5 Verilog Code **5**

5.1 PIPO Shift Register 5

5.2 PISO Shift Register 5

6 Test Bench **6**

6.1 PIPO Test Bench 6

6.2 PISO Test Bench 8

7 Output **10**

7.1 Waveforms 10

7.2 Console Output 10

8 References **11**

9 GitHub Repository **12**

1. Problem Statement

Design and implement a 4-bit shift register (Parallel-In Parallel-Out and Parallel-In Serial-Out) using Verilog HDL, and verify it using a test bench.

Team Members:

- Rahul P – PES1UG25AM510
- Rothith M – PES1UG25AM512
- S Banuteja Reddy – PES1UG25AM513
- Sambhram Laxman Sattigeri – PES1UG25AM516

2. Theory

2.1 Shift Registers

A shift register is a group of flip-flops connected in series to store and shift multiple bits of data either left or right with clock pulses.

The two types of shift registers based on the direction of data movement are:

- **Shift Left Register:** A register that shifts the stored bits towards the left with each clock pulse.
- **Shift Right Register:** A register that shifts the stored bits towards the right with each clock pulse.

2.2 PIPO Shift Register

A Parallel-In Parallel-Out (PIPO) shift register accepts and outputs data simultaneously across all flip-flops without any serial shifting, using synchronized control signals. It is used as a temporary storage device and also acts as a delay element.

The main characteristics of a PIPO shift register are:

- Data is loaded and retrieved in parallel from each flip-flop at the same time.
- It consists of D flip-flops with no serial interconnections between them, controlled by a common clock and clear (CLR) signals.

Working: On every rising clock edge, if `load` is high, the 4-bit input `data_in` is transferred to `data_out` in a single clock cycle. If `load` is low, the register retains its previous value.

2.3 PISO Shift Register

A Parallel-In Serial-Out (PISO) shift register accepts parallel data simultaneously into each flip-flop and shifts it out serially through a single output line using synchronized flip-flops and multiplexers. It is used to convert parallel data into serial data.

The main characteristics of a PISO shift register are:

- Data is loaded in parallel and then shifted out one bit at a time serially.
- It consists of D flip-flops with inputs controlled by multiplexers that select between parallel data and the output of the previous flip-flop. All flip-flops operate using a common clock signal.

Working: On every rising clock edge, if `shift` is low, `parallel_in` is loaded into the internal register `temp`. If `shift` is high, the contents of `temp` move one position to the right and a 0 enters the most significant bit. The least significant bit, `temp[0]`, is available on `serial_out`, so the loaded data leaves serially, LSB first, over four clock cycles.

3. Circuit

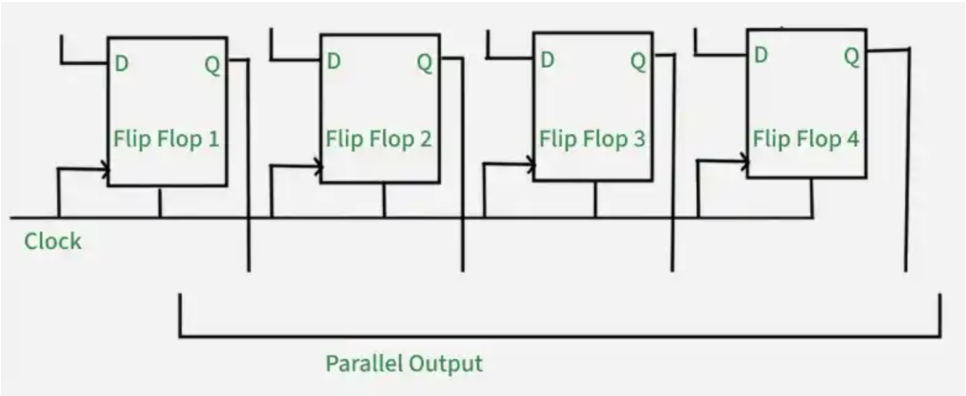


Figure 1: Logic diagram of 4-bit PIPO shift register

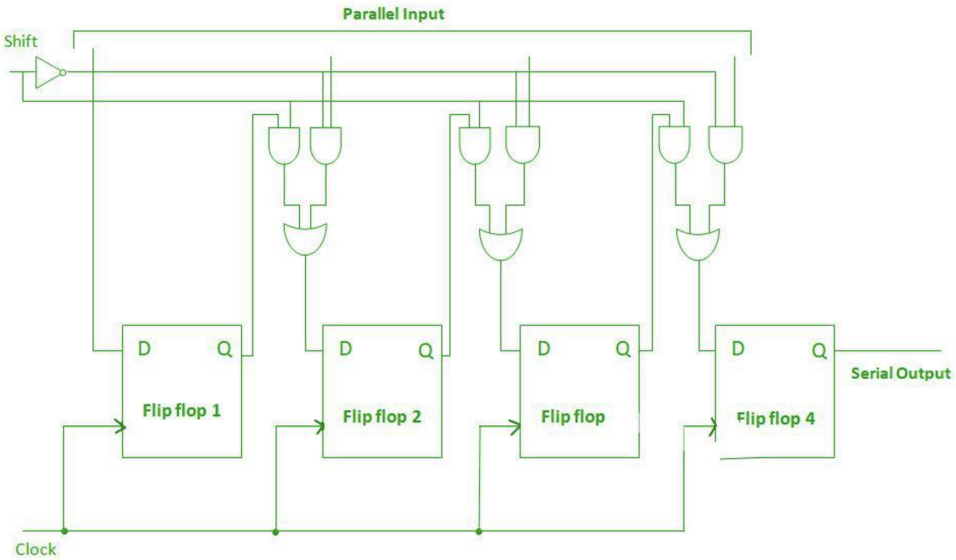


Figure 2: Logic diagram of 4-bit PISO shift register

4. Truth Table

Table 1: Function table of 4-bit PIPO shift register

clk	load	data_out[3:0]
↑	1	data_in[3:0]
↑	0	No change

Table 2: Function table of 4-bit PISO shift register

clk	shift	Operation
↑	0	Parallel load: temp ← parallel_in
↑	1	Shift right: temp ← {0, temp[3:1]}

Table 3: PISO shift sequence for parallel_in = 1010

Clock	shift	temp[3:0]	serial_out
Load	0	1010	0
1	1	0101	1
2	1	0010	0
3	1	0001	1
4	1	0000	0

5. Verilog Code

5.1 PIPO Shift Register

```
module PIPO_Shift_Register(
    input wire clk,
    input wire load,
    input wire [3:0]data_in,
    output reg [3:0]data_out
);

    always @(posedge clk) begin
        if (load)
            data_out <= data_in;
    end

endmodule
```

5.2 PISO Shift Register

```
module PISO_Shift_Register (
    input wire clk,
    input wire shift,
    input wire [3:0] parallel_in,
    output wire serial_out
);
```

```
    reg [3:0] temp;

    always @(posedge clk) begin
        if (!shift)
            temp <= parallel_in; // Parallel Loading
        else
            temp <= {1'b0, temp[3:1]}; // Shift Towards Right
    end

    assign serial_out = temp[0];

endmodule
```

6. Test Bench

6.1 PIPO Test Bench

```
module PIPO_Shift_Register_tb;

    reg clk;
    reg load;
    reg [3:0] data_in;
    wire [3:0] data_out;

    // Instantiate the PIPO Shift Register
    PIPO_Shift_Register test (
        .clk(clk),
        .load(load),
        .data_in(data_in),
        .data_out(data_out)
    );

    // Clock generation
    always begin
        #10 clk = ~clk;
    end

    initial begin

        // Dump waveform
        $dumpfile("pipo_wave.vcd");
        $dumpvars(0, PIPO_Shift_Register_tb);

        // Initialize inputs
        clk = 0;
        load = 0;
        data_in = 4'b0000;

        $display("==== PIPO Shift Register Simulation ====");
        $display("Initial: Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b",
            $time, clk, load, data_in, data_out);

        // Test Case 1: Load 1010
        #10;
```

```
        load = 1;
        data_in = 4'b1010;
        $display("Test Case 1 (Load 1010)    : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);

        // Test Case 2: Load 1100
        #10;
        $display("Result of Test Case 1      : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);
        data_in = 4'b1100;
        $display("Test Case 2 (Load 1100)    : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);

        // Test Case 3: Disable loading
        #10;
        $display("Result of Test Case 2      : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);
        load = 0;
        data_in = 4'b0110;
        $display("Test Case 3 (Hold, load=0) : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);

        // Test Case 4: Load 1111
        #10;
        $display("Result of Test Case 3      : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);
        load = 1;
        data_in = 4'b1111;
        $display("Test Case 4 (Load 1111)    : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);

        // Test Case 5: Disable loading
        #10;
        $display("Result of Test Case 4      : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);
        load = 0;
        data_in = 4'b0000;
        $display("Test Case 5 (Hold, load=0) : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);

        #10;
        $display("Result of Test Case 5      : Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);
        $display("==== Simulation Finished ====");
        $finish;

    end

    // Monitor input and output values
    initial begin
        $display("Time=%0t | clk=%b | load=%b | data_in=%b | data_out=%b", $time, clk, load, data_in, data_out);
    end

endmodule
```

6.2 PISO Test Bench

```
module PISO_Shift_Register_tb;

    reg clk;
    reg shift;
    reg [3:0] parallel_in;
    wire serial_out;

    PISO_Shift_Register test (
        .clk(clk),
        .shift(shift),
        .parallel_in(parallel_in),
        .serial_out(serial_out)
    );

    always begin
       #10 clk = ~clk;
    end

    initial begin

        $dumpfile("piso_wave.vcd");
        $dumpvars(0, PISO_Shift_Register_tb);

        clk = 0;
        shift = 0;
        parallel_in = 4'b0000;

        $display("==== PISO Shift Register Simulation ====");
        $display("Initial: Time=%0t | clk=%b | shift=%b | parallel_in=%b | serial_out=%b
            ", $time, clk, shift, parallel_in, serial_out);

        // Parallel loading
        @(negedge clk);
        parallel_in = 4'b1010;
        shift = 0;
        $display("Test Case 1 (Load 1010)    : Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | serial_out=%b", $time, clk, shift, parallel_in, serial_out)
            ;

        @(posedge clk);
        #1;
        $display("Parallel Loaded: %b", test.temp);
        $display("Result of Test Case 1      : Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | temp=%b | serial_out=%b", $time, clk, shift, parallel_in,
            test.temp, serial_out);

        // First shift
        @(negedge clk);
        shift = 1;
        $display("Test Case 2 (Shift enabled): Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | serial_out=%b", $time, clk, shift, parallel_in, serial_out)
            ;

        @(posedge clk);
        #1;
        $display("After 1st shift: %b | Serial: %b",
```

```
                test.temp, serial_out);
        $display("Result of 1st shift        : Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | temp=%b | serial_out=%b", $time, clk, shift, parallel_in,
            test.temp, serial_out);

        // Second shift
        @(posedge clk);
        #1;
        $display("After 2nd shift: %b | Serial: %b",
                test.temp, serial_out);
        $display("Result of 2nd shift        : Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | temp=%b | serial_out=%b", $time, clk, shift, parallel_in,
            test.temp, serial_out);

        // Third shift
        @(posedge clk);
        #1;
        $display("After 3rd shift: %b | Serial: %b",
                test.temp, serial_out);
        $display("Result of 3rd shift        : Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | temp=%b | serial_out=%b", $time, clk, shift, parallel_in,
            test.temp, serial_out);

        // Fourth shift
        @(posedge clk);
        #1;
        $display("After 4th shift: %b | Serial: %b",
                test.temp, serial_out);
        $display("Result of 4th shift        : Time=%0t | clk=%b | shift=%b |
            parallel_in=%b | temp=%b | serial_out=%b", $time, clk, shift, parallel_in,
            test.temp, serial_out);

        #10;
        $display("==== Simulation Finished ====");
        $finish;

    end

endmodule
```

7. Output

7.1 Waveforms

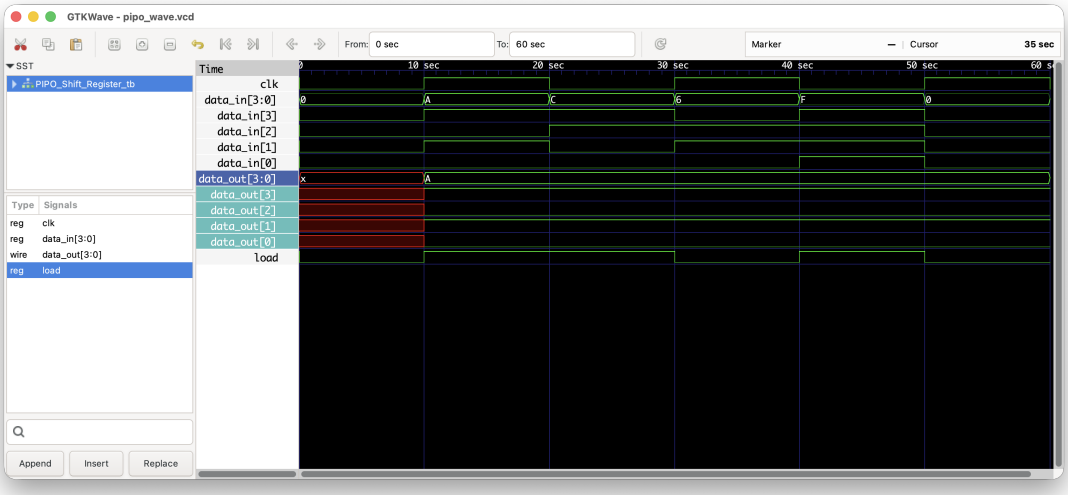


Figure 3: PIPO simulation waveform

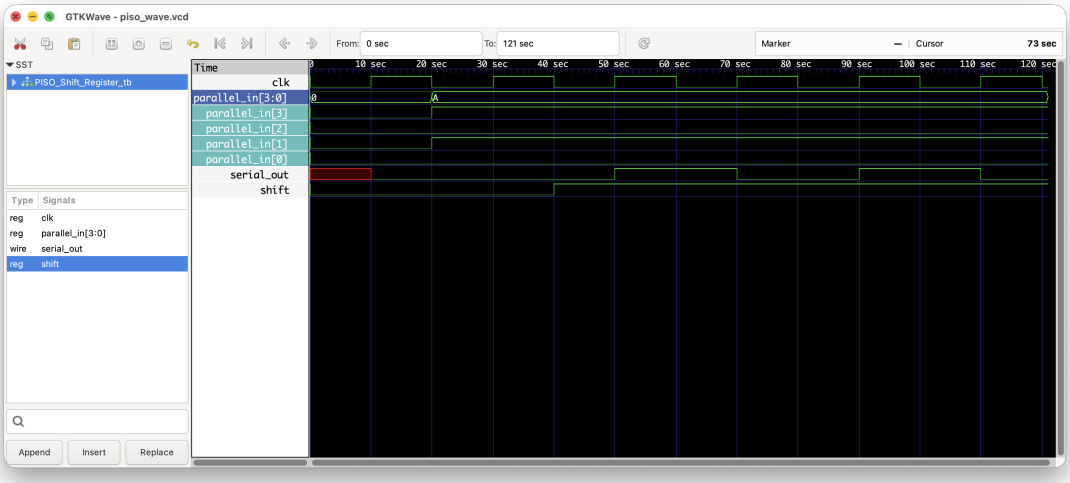


Figure 4: PISO simulation waveform

7.2 Console Output

```
==== PIPO Shift Register Simulation ====
Initial: Time=0 | clk=0 | load=0 | data_in=0000 | data_out=xxxx
Time=0 | clk=0 | load=0 | data_in=0000 | data_out=xxxx
Test Case 1 (Load 1010)     : Time=10 | clk=1 | load=1 | data_in=1010 | data_out=xxxx
Result of Test Case 1       : Time=20 | clk=0 | load=1 | data_in=1010 | data_out=1010
```

```
Test Case 2 (Load 1100)    : Time=20 | clk=0 | load=1 | data_in=1100 | data_out=1010
Result of Test Case 2      : Time=30 | clk=1 | load=1 | data_in=1100 | data_out=1010
Test Case 3 (Hold, load=0) : Time=30 | clk=1 | load=0 | data_in=0110 | data_out=1010
Result of Test Case 3      : Time=40 | clk=0 | load=0 | data_in=0110 | data_out=1010
Test Case 4 (Load 1111)    : Time=40 | clk=0 | load=1 | data_in=1111 | data_out=1010
Result of Test Case 4      : Time=50 | clk=1 | load=1 | data_in=1111 | data_out=1010
Test Case 5 (Hold, load=0) : Time=50 | clk=1 | load=0 | data_in=0000 | data_out=1010
Result of Test Case 5      : Time=60 | clk=0 | load=0 | data_in=0000 | data_out=1010
==== Simulation Finished ====

==== PISO Shift Register Simulation ====
Initial: Time=0 | clk=0 | shift=0 | parallel_in=0000 | serial_out=x
Test Case 1 (Load 1010)    : Time=20 | clk=0 | shift=0 | parallel_in=1010 | serial_out=0
Parallel Loaded: 1010
Result of Test Case 1      : Time=31 | clk=1 | shift=0 | parallel_in=1010 | temp=1010 |
    serial_out=0
Test Case 2 (Shift enabled): Time=40 | clk=0 | shift=1 | parallel_in=1010 | serial_out=0
After 1st shift: 0101 | Serial: 1
Result of 1st shift        : Time=51 | clk=1 | shift=1 | parallel_in=1010 | temp=0101 |
    serial_out=1
After 2nd shift: 0010 | Serial: 0
Result of 2nd shift        : Time=71 | clk=1 | shift=1 | parallel_in=1010 | temp=0010 |
    serial_out=0
After 3rd shift: 0001 | Serial: 1
Result of 3rd shift        : Time=91 | clk=1 | shift=1 | parallel_in=1010 | temp=0001 |
    serial_out=1
After 4th shift: 0000 | Serial: 0
Result of 4th shift        : Time=111 | clk=1 | shift=1 | parallel_in=1010 | temp=0000 |
     serial_out=0
==== Simulation Finished ====
```

8. References

References

[1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2012.

[2] GeeksforGeeks, "Shift Registers in Digital Electronics." [Online]. Available: https://www.geeksforgeeks.org/digital-logic/shift-registers-in-digital-electronics/. [Accessed: 11-Nov-2026].

9. GitHub Repository

Complete source code, test benches, waveforms and simulation logs are available at:
`https://github.com/Sambhram1087/pipo_piso_shift_registers`